Nuclear Central Manager

Workshop Exercise

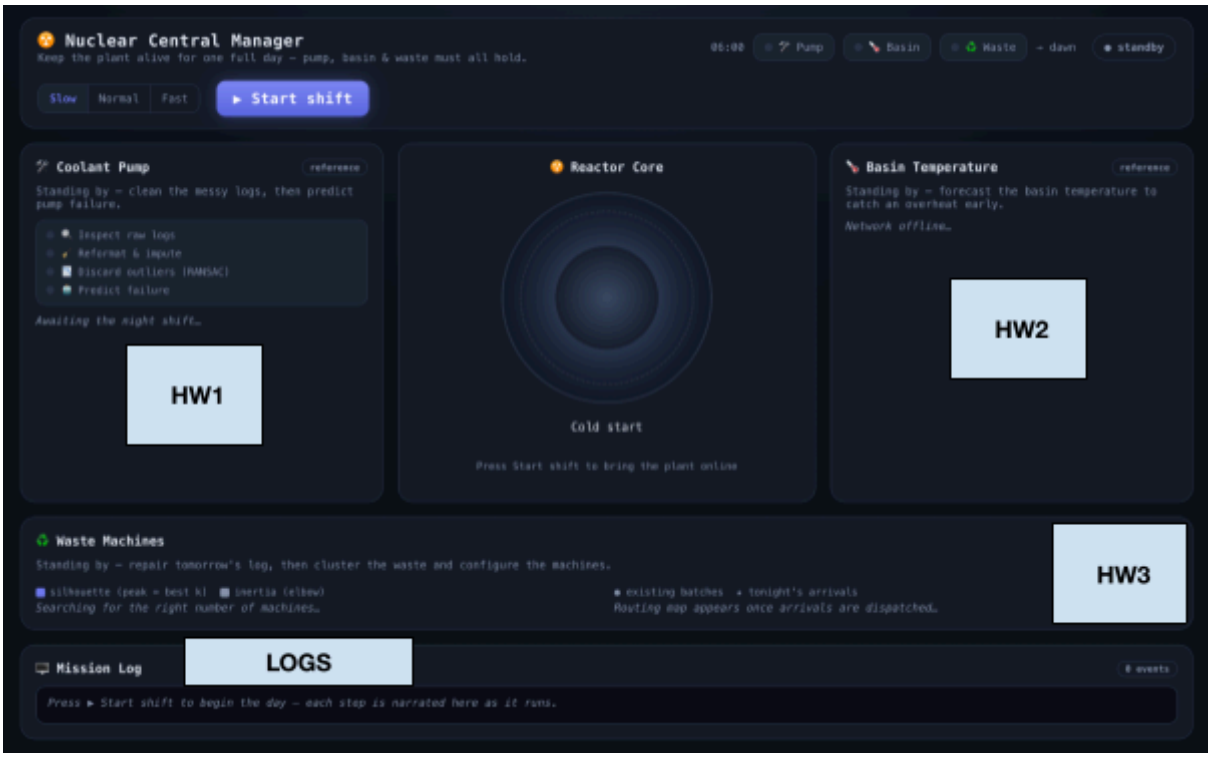
From Data to Decisions — FS26

ETH Zürich

PART 1

Context

You are the newly appointed manager of a nuclear power plant. Your first shift starts now! Three critical subsystems stand between you and a meltdown: the coolant pump must be monitored and its failures predicted before the core overheats (HW1), the basin temperature must be forecast in real time because the primary sensors are failing under radiation (HW2), and tonight's radioactive waste batches must be clustered and routed to the right machines before the night shift begins (HW3).
All three of your solutions run together inside the control panel. If any one of them fails its safety check, the plant goes down.



Welcome to the closing exercise of the week! This exercise brings together the core skills you have developed throughout the course into a single, integrated application: the **Nuclear Central Manager**. You will work through three Colab notebooks, each addressing a distinct component of a simulated nuclear power plant's control system. Once you have correctly completed the solutions, you will download the resulting Python files and plug them into a live web application that visualises the plant's operational status in real time.

Each notebook produces a self-contained solution module. Together, the three modules power the plant's control panel: one monitors the coolant pump, another predicts basin temperature, and the

third configures waste-processing machines. The sections below explain what each notebook covers and why it matters.

Notebook 1 — Pump Failure Prediction

HW1_pump_cleaning_fitting_solution.ipynb

The coolant pump is the most critical component in the plant. If it seizes without warning, the reactor core overheats. In this notebook you receive a messy sensor log from the pump and must clean it thoroughly before training a classifier that predicts pump failures. The data contains formatting inconsistencies, missing readings, duplicates, and physical outliers — all of which must be addressed before any model can be trusted.

Key techniques include IQR-based outlier detection, median imputation, RANSAC fitting to remove physically implausible rows, and logistic regression with strategies for handling class imbalance (random oversampling, class-weighted loss). You will evaluate your model using precision, recall, and F1-score, because raw accuracy is misleading when failures are rare.

The output of this notebook is a `PumpSolution` object that the live control panel calls on every new batch of telemetry. It must achieve both a recall threshold (catch real failures) and a precision floor (avoid flooding the operator with false alarms) to keep the plant running safely.

Notebook 2 — Basin Temperature Prediction

HW2_basin_temperature_solution.ipynb

The basin's primary temperature sensors are degrading under radiation exposure. This notebook asks you to complete a multi-modal neural network that serves as a backup predictor. You inherit a half-built PyTorch architecture from a (fictional) burned-out engineer and must finish four independent blocks: extracting signal features from a 1-D CNN, reshaping thermal camera tensors for a frozen vision encoder, implementing a residual MLP with a skip connection, and adding a regression head that outputs a single temperature value in °C.

Along the way, you will practise tensor manipulation with `einops`, transfer learning (slicing and freezing a pre-built encoder), and multi-modal fusion via feature concatenation. The model is trained with MSE loss and compared against a predict-the-mean baseline.

The control panel calls your trained model on live secondary sensor streams and thermal camera frames. If the forecast error is too large or the model misses an overheat event, the simulation triggers a meltdown.

Notebook 3 — Waste Batch Clustering

HW3_clustering_waste_solution.ipynb

Every night, waste-processing machines must be configured before the shift starts, and a machine can only safely handle waste that matches its settings. Without clustering, each of hundreds of batches would need a hand-tuned configuration — operationally impossible. In this notebook you build a full unsupervised pipeline that groups radioactive waste batches into clusters, where each cluster maps to one machine configuration (shielding level, temperature setting, liner type).

You will select informative features, apply log-transforms for heavy-tailed distributions (radioactivity, half-life), use a `ColumnTransformer` pipeline for heterogeneous preprocessing, and determine the optimal number of clusters using silhouette scores and the inertia elbow method. The pipeline is extended to route new arriving batches into existing clusters and to repair incomplete intake logs using supervised Random Forest models.

The notebook produces three reusable functions — `build_machine_plan`, `route`, and `fill_gaps` — that the control panel calls to set up tonight's machines, dispatch new truck arrivals, and repair tomorrow's gappy log before re-planning.

PART 2

Setup Instructions

Follow the steps below to clone the repository, install dependencies, and launch the application. All commands are provided for both macOS/Linux and Windows (PowerShell).

Step 1 — Prerequisites

Installing npm

Node.js ships with `npm` (Node Package Manager) included. Installing Node.js automatically gives you both. Below are the recommended installation steps for macOS/Linux and Windows.

macOS / Linux

Option A — Official installer (simplest)

Download the LTS installer from **https://nodejs.org** and follow the on-screen instructions. This installs both `node` and `npm` system-wide.

Option B — Homebrew (macOS)

```
brew install node
```

Windows

Option A — Official installer (simplest)

Download the LTS `.msi` installer from **https://nodejs.org** and run it. Accept the defaults — the installer adds `node` and `npm` to your PATH automatically.

Rest Prerequisites

Make sure the following tools are installed on your machine before proceeding:

Tool	Required?	Purpose
Python ≥ 3.10	Yes	Backend (FastAPI, scikit-learn, PyTorch)
git	Yes	Clone the repository
uv	Recommended	One-command environment setup
pip	Alternative to uv	Manual dependency installation

Installing Python

Installation Guide

Installing git

Installation Guide

Installing uv

Documentation

macOS / Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
# or with Homebrew:
brew install uv
```

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

> 💡 After installing `uv`, open a new terminal window so the command is available on your PATH.

Installing pip

Installation Guide

Step 2 — Clone the Repository

```
git clone https://github.com/eth-fdd-fs26/FDD-WE0-public.git
```

Then navigate into the repository root. All subsequent commands assume you are in this directory.

```
cd FDD-WE0-public
```

Step 3 — Install Dependencies

Option A — With uv (recommended)

No manual installation needed. When you launch the app for the first time (Step 6), `uv` automatically creates an isolated environment and installs all dependencies. Skip ahead to Step 4.

Option B — With pip

```
pip install -r workshop/requirements.txt
```

This installs FastAPI, uvicorn, scikit-learn, PyTorch, joblib, and all other required packages.

Step 4 — Place Your Solution Files

After completing each notebook in Google Colab, run the final cell to export your solution as a Python file (along with any trained model artifacts). Place these files in the following directory:

```
workshop/solutions/
```

Notebook	File to Place	Additional Artifacts
HW1 — Pump Classifier	`solution_part1.py`	`pump_model.joblib` (if any)
HW2 — Basin Temperature	`solution_part2.py`	Trained model file
HW3 — Waste Clustering	`solution_part3.py`	Any `.joblib` / `.pkl` files

Example folder structure:

```
FDD-WE0-public/
└── workshop/
    └── solutions/
        ├── solution_part1.py
        ├── pump_model.joblib
        ├── solution_part2.py
        ├── basin_net.pt
        └── solution_part3.py
```

Step 5 — Build the Frontend (Optional)

The pre-built frontend is already included in the repository (`workshop/frontend/dist/`), so you can **skip this step entirely**. Only run the commands below if you have edited the React source files.

macOS / Linux:

```
cd workshop/frontend
npm install
npm run build
cd ../..
```

Windows (PowerShell):

```
cd workshop\frontend
npm install
npm run build
cd ..\..
```

Step 6 — Launch the Application

All launch commands are run from the repository root.

With uv (recommended)

```
uv run --project workshop python workshop/launch.py
```

On first run, `uv` builds an isolated Python environment automatically (this may take around 30 seconds), then starts the server.

With pip

macOS / Linux:

```
python workshop/launch.py
```

Windows (PowerShell):

```
python workshop\launch.py
```

The launcher prints the URL and opens your browser automatically. Press Ctrl+C to stop the server.

Step 7 — Run Modes

The application supports three run modes, giving you flexibility to test your solutions incrementally.

a) Full Reference Mode

Use this to see the complete working application **before** you have finished any notebook. No solution files are needed.

```
# with uv:
uv run --project workshop python workshop/launch.py --allow-fallback

# with pip:
python workshop/launch.py --allow-fallback
```

Every subsystem runs the bundled reference solution. Panel badges show **reference**.

b) Full Custom Mode

Place all three `solution_part*.py` files (plus their artifacts) into `workshop/solutions/`, then launch **without** the `--allow-fallback` flag:

```
# with uv:
uv run --project workshop python workshop/launch.py

# with pip:
python workshop/launch.py
```

Panel badges show **student**. If any solution file is missing, the plant immediately melts down.

c) Mixed Mode

Place whichever notebook solutions you have completed so far, then launch with `--allow-fallback`. The reference fills in for any missing parts automatically:

```
# with uv:
uv run --project workshop python workshop/launch.py --allow-fallback

# with pip:
python workshop/launch.py --allow-fallback
```

Subsystems with a solution file run **your code**; those without one run the **reference** instead. This is ideal for testing one homework at a time during development.

> 💡 You can test each part independently. For example, place only `solution_part1.py` and run with `--allow-fallback` to verify your pump classifier while the other two subsystems use the reference.

Step 8 — Access the Application

Open your browser and navigate to:

```
http://127.0.0.1:8000
```

The launcher opens this URL automatically. If port 8000 is already in use, the launcher picks the next free port and prints the actual URL in the terminal. Press **Start Shift** to begin the day simulation and watch the control panel come to life.

Quick Reference

Goal	Command (from repo root)
Launch with reference (demo)	`uv run --project workshop python workshop/launch.py --allow-fallback`
Launch with your solutions	`uv run --project workshop python workshop/launch.py`
Launch mixed mode	`uv run --project workshop python workshop/launch.py --allow-fallback`
Install deps without uv	`pip install -r workshop/requirements.txt`

Rebuild frontend (only if you edit the UI)	`cd workshop/frontend && npm install && npm run build`

Good luck, and happy coding!